1. Project Overview

We are building Firefly to assist remote helpers in effectively communicating difficult technology tasks to learners by creating a real-time highlighting transcription service that turns instructions from voice and video calls into visual guidance displayed on screens, camera views, and AR glasses, without ever taking control of the learner's device.

2. Scope

2.1 In Scope

SYS-01: Speech Capture and Transcription

a. Capture call audio from the microphone or on loopback, clean it up, detect speech, and transcribe it locally with some kind of model.

SYS-02: Instruction Boundary Detection

a. Detect voice and know when an instruction is given by the helper and capture what the instruction is

SYS-03: On-Screen Object Detection

a. Generate coordinates for certain objects based on screenshot/image and given instruction by the helper

SYS-04: Highlight Rendering and Animations

a. Take coordinate input and generate a highlight animation to display instruction to user using a language model

SYS-05: Real Time Image Processing and Display

a. Evaluate real-time objects on camera/screen with OpenCV, and display the highlighting/animations on the camera viewing screen using Windows webcam

2.2 Under Evaluation

SYS-06: iOS app

a. Create a mobile app supported on iOS to support camera detection through an iPhone device.
    i. This is under evaluation as it is another operating system for our system to use, which will require special software and testing. The app will be in/out of scope depending on how confident we are with our in scope requirements and how much time we have to devote to its development.

SYS-07: MacOS/Android

a. Support for Mac and Android users to expand the user base outside of only iPhone and Windows users.
    i. Similar to the iOS app, this will require special software development to apply our project to other OS's. If we decide we have the time and knowledge to achieve the new operating systems, this will become an in-scope objective

SYS-08: AR Glasses

a. Utilize AR glasses technology to provide users with a commercial and lightweight alternative to using a phone/camera.
    i. The challenge with AR glasses is the price and limited development software currently available for the glasses. In our brief research, publicly available AR glasses are either out of budget, or will likely have too limited of an SDK to develop what we want on it. If we research more and believe it is feasible, we will pursue this requirement.

SYS-09: Persistent instruction storage + instruction playback

a. Store previous instructions and locations of past instructions for the AI call to utilize in the future. This would provide faster instructions to the helper and assist the teacher with common problems.
    i. This requirement was deemed to be not necessary, as normal API calls will more than likely work as intended. However, if we want to improve run times or have time to improve our project further near the end, we would work on implementing a save system on the device.

2.3 Out of Scope

1. AI teacher
    a. Creating a complete AI teacher will not be in scope, as we wanted to preserve the human side of teaching and interactions with others. We also decided it would be too complex to develop an AI teacher that could teach anything to anyone, so this will remain out of scope
2. Ability to teach to multiple screens/students at once
    a. Having multiple learners being taught by one helper using our project is something we could expand on in the far future, but it was decided that this will remain out of scope for this project as it is too complex and will require more testing and time than we have.
3. Remote Control
    a. We do not want a remote controlled system by both the AI and teacher, as we believe it would take away from what the learner is truly learning. We believe that a learner should achieve a task themselves to properly learn a skill, so any kind of remote control will remain out of scope
4. Autonomous Task Completion
    a. Similar to remote control, we do not want the AI to complete tasks for the user. We instead want the user to complete tasks on their own for their own learning sake. Therefore, autonomous task completion will be out of scope.
5. Building our own calling platform
    a. Building our own calling platform is too complex for us to achieve in one year. There are already multiple calling platforms that we can utilize commercially, so we will keep any custom platforms outside of our project scope.

2.4 Assumptions

1. Stable Internet Connection
2. Windows Computer/ iPhone
3. Customer (learner) does not have complex technological understanding
    a. User needs to install app easy
4. User (teacher) audio must have technological understanding and must have clear audio
5. User (teacher) speaks English
6. Some personal information may be captured however not stored when detecting on screen/on camera details.

2.5 Constraints

1. Instruction visual transcription speed will be limited by available models
2. AR glasses have limited development kits and availability is limited by budget
3. Phone and AR compute and battery limit on-device processing

3. Requirements Elicitation and Prioritization

Submitted on Brightspace

4. Software Requirements

FR-01

- **SRID**: SYS-01
- **ID:** FR-01
- **Title:** Audio Input Capture
- **Statement:** When the learner enables Firefly, Firefly needs to capture call audio from the microphone or system loopback from an app on the computer.
- **Rationale:** Firefly has to be able to hear the helper through the calling platform that is used, such as Zoom, FaceTime, or a phone call, while being independent from the call itself.
- **Test method:** Play a known 30 s recording over Zoom, FaceTime, and a phone call at least 10 times. At least 95% of the recording's duration must be captured on each loop, with audio processable by next steps.
- **Supporting context:** Loopback is audio that is playing on the computer being captured. Call audio is the helper's voice caught through the microphone.
- **Tracing:** SYS-01
- **Priority:** Must Have

FR-02

- **SRID:** SYS-01

- **ID:** FR-02
- **Title:** Audio Pre-Processing
- **Statement:** When audio is captured actively, Firefly needs to suppress echo and background noise, as well as detect the start and end of someone speaking before performing transcription within 300ms of each tail.
- **Rationale:** Calls could contain echo and a lot of background noise, which lower accuracy of transcription. Silence and noise should not be sent to the transcriber to reduce latency.
- **Test method:** Record 10 scripted commands with constant background noise or DSP generate background noise. Run each recording through the pre-processing step, then pass the result to the transcription step. The test passes if 1) at least 8 of the 10 commands are transcribed with the action and target intact, and 2) speech detection step marks a start and an end for each command within 300 ms, with none cut off mid-sentence.
- **Supporting context:** Word error rate will be the share of words transcribed wrongly. Echo is the helper's voice returning through the learner's speakers. Speech detection aka voice activity detection marks where speech begins and ends.
- **Tracing**: SYS-01
- **Priority**: Must Have

FR-03
- **SRID:** SYS-01
- **ID:** FR-03
- **Title:** Speech Transcription
- **Statement:** When speech is detected, Firefly will convert the speech to text that can be passed into the model within seconds of the speech ending.
- **Rationale**: Transcription will allow a spoken instruction to be processed by the system. Running it locally keeps raw audio on the learner's machine, which supports privacy for the user.
- **Test method:** Transcribe 50 predefined commands such "click the print button" and require a word error rate of 15% or less. Record the total time from audio to transcription and ensure that the delay is within 2 seconds.
- **Supporting context**: On the learner's device means the transcription model runs locally rather than in the cloud. Speech detected refers to the output of FR-02.
- **Tracing**: SYS-01
- **Priority**: Must Have

FR-04
- **SRID**: SYS-02
- **ID**: FR-04
- **Title**: Instruction Identification

- **Statement**: After a transcript is created, Firefly needs to classify it as an applicable instruction or as background talk within 1 second, and only start target identification for transcripts that are instructions.
- **Rationale**: Helpers and learners will talk normally during a call, nor just give instructions. Firefly must be able to distinguish between real instructions to act upon vs small talk to not trigger random highlights and cloud requests, which could add cost and be confusing to the learner.
- **Test Method**: Run 40 labeled utterances through the classifier with 20 instructions and 20 small talk lines. Verify at least 18 of 20 instructions are accepted and at least 18 of 20 conversational lines are rejected. Also feed 10 whole conversations, and have Firefly return all instructions given in each conversation.
- **Supporting Context:** An instruction is a statement that defines an action and a target on the screen or in the room. Conversation is any speech that does not do so and is irrelevant to Firefly.
- **Trace:** SYS-02 Instruction Boundary Detection
- **Priority**: Must Have

FR-05
- **SRID**: SYS-02
- **ID**: FR-05
- **Title**: Instruction Boundary Detection
- **Statement**: When the speaker has been silent for 0.5 second after an instruction, Firefly will identify the instruction as complete and start processing.
- **Rationale**: Sending a half finished instruction gives the wrong intention to Firefly, and waiting too long slows the whole pipeline. A fixed pause is a simple, testable signal that the helper is done with a particular instruction.
- **Test Method**: Record 20 spoken instructions with natural pauses and at least 18 have correct instructions returned fully, and none are split into two instructions. Then, record 5 instructions with a short mid-sentence pause under 0.2 seconds and confirm they are not split.
- **Supporting Context**: Silent means no speech detected by the speech detection step in FR-02. Mark complete means the text is passed on and the pipeline and screenshotting begin. The 0.5 second pause is a starting value subject to tuning
- **Trace**: SYS-02 Instruction Boundary Detection
- **Priority**: Must Have

FR-06
- **SRID**: SYS-02
- **ID**: FR-06
- **Title**: Sequential Instruction Processing

- **Statement**: When an instruction contains multiple steps or the target is not visible in the current view of the screen, Firefly will break the instruction into a list of steps and show each step's highlight sequentially after each step has been completed.
- **Rationale**: Tasks such as paying a bill take several steps, and a helper often names the final goal ("open the print menu") when the first needed element, such as a File menu, must be clicked first because they may not be able to see the screen. The learner needs to have clear 1 by 1 steps.
- **Test Method:** Give 10 scripted multi-step instructions such as "click File, then Print, then OK" and verify the highlights appear in the spoken order, one at a time. Give 10 instructions whose target is hidden behind a menu and verify that the first highlight is on the menu item that must be opened first in at least 4 of 5. In all tests, the next highlight should only show after FR-12(Completion) is detected.
- **Supporting Context:** A step is one highlight the learner acts on. Completion is defined in FR-12 (Completion Detection). Not visible means the element is not on the captured current screen as it could be inside a closed menu or on another page.
- **Trace**: SYS-02 Instruction Boundary Detection
- **Priority**: Should Have

FR-07

- **SRID**: SYS-03
- **ID**: FR-07
- **Title**: Capture Field of View
- **Statement**: When an instruction is completed by the helper, Firefly should capture exactly one screenshot or camera frame within 100 ms.
- **Rationale**: The image is what allows the system to find the target to highlight. Capturing should only be when an instruction is given, preserving as much of the learner's privacy as possible to keep cloud costs down.
- **Test Method:** Run a 5-minute session with 3 spoken instructions and 2 conversational remarks. Verify exactly 3 captures occur, each within 100 ms of its instruction being marked complete, and none during the conversation or silence. Also test failures with a lock screen or black screen 5 times.
- **Supporting Context:** A field of view is whatever the learner is looking at such as the current screen of camera view that is on the computer. Marked complete refers to the output of FR-05.
- **Trace**: SYS-03 Target Identification
- **Priority**: Must Have

FR-08

- **SRID**: SYS-03
- **ID**: FR-08
- **Title**: Target Identification

- **Statement**: When an instruction and an image taken, Firefly will send the elements to an AI model to identify the element the instruction refers to, and shall return the result within 8 seconds in at least 90% of requests with 90% accuracy.
- **Rationale**: Finding the correct element from a spoken description and a picture is the core function of Firefly. Without a correct target, the highlight would mislead the learner.
- **Test Method**: Prepare 30 screenshots of common tasks (for example a printer dialog, a bill-payment page, a settings menu), each with a spoken instruction and a known correct element. Verify the model identifies the correct element in at least 27 of 30, and that at least 27 of 30 responses arrive within 8 seconds.
- **Supporting Context:** The model takes an image and text together as input. The correct element means the box returned overlaps the true element (intersection overlap of at least 0.5, defined in FR-09).
- **Trace**: SYS-03 Target Identification
- **Priority**: Must Have

FR-09
- **SRID**: SYS-03
- **ID**: FR-09
- **Title**: Bounding Box Return
- **Statement**: When a target is identified by the AI model, Firefly returns it as a bounding box in normalized coordinates that has at least 50% intersection. If the box coordinates are malformed, the confidence is below 70%, or no response arrives within 10 seconds, Firefly won't show any highlight and display a short message that the target could not be found.
- **Rationale**: A wrong highlight is worse than none for a novice user, so the system must fail safely and let the helper rephrase. The bounding box allows an accurate highlight.
- **Test Method:** Check 20 different returned boxes against hand-marked coordinates and verify that at least 16 (80%) have an intersection over union of 0.5 or more. Feed 5 malformed responses, 5 low-confidence results, and 5 simulated timeouts. In all 15 cases, no highlight is drawn and a message appears.
- **Supporting Context**: A bounding box is a rectangle given as left, top, width, and height. Normalized means each value is a fraction of the image size from 0 to 1. Intersection is the overlap area divided by the combined area of the returned and actual boxes.
- **Trace**: SYS-03 Target Identification
- **Priority**: Must Have

FR-10
- **SRID**: SYS-04
- **ID**: FR-10
- **Title**: Highlight Overlay Rendering

- **Statement**: When a valid bounding box is received, Firefly draws a highlight on top of the target, converting returned normalized coordinates to the learner's display resolution.
- **Rationale**: The highlight is the only guidance that the learner sees, so it must be in the correct place and does not appear late.
- **Test Method:** Display 10 known targets such as a button at three settings: 1080p at 100% scaling, 1440p at 125%, and 4K at 150%. For each, send a bounding box for the target and measure the distance from the highlight's center to the target's center. Verify that 27 of 30 are within 10 pixels of the center of the highlight. Send malformed coordinates and boxes that are off-screen to ensure that they return errors rather than try displaying something.
- **Supporting Context:** A highlight is a visible glowing outline on top of the screen. Target's center is the middle of the true element.
- **Trace**: SYS-04 Highlight Rendering
- **Priority**: Must Have

FR-11
- **SRID**: SYS-04
- **ID**: FR-11
- **Title**: Non-Interactive Overlay
- **Statement**: While a highlight is displayed, the overlay should not interfere with the learner's mouse, keyboard, and touch input and should not send any mouse, keyboard, or touch input to any device.
- **Rationale**: The learner must stay in full control of their device so that they can perform every function, and so that there are no strange behaviors with the inputs. Firefly is not designed for remote control.
- **Test Method:** With a highlight displayed, interact with it(scroll, button press, text field), and verify that there is no interference 10/10 times. Then run 10 five minute sessions with instructions, and log input events at the OS level to verify that every event came from the learner's own hardware and none from Firefly.
- **Supporting Context:** A non-interactive overlay is a transparent window that shows the highlight but is transparent in interaction. Input injection means software performing presses on its own, which should not happen.
- **Trace:** SYS-04 Non-Interactive Overlay
- **Priority:** Must Have

FR-12
- **SRID**: SYS-04
- **ID**: FR-12
- **Title**: Completion Detection

- **Statement**: When the learner successfully clicks inside the highlight, dismisses the highlight, or a timeout of 15 seconds passes, Firefly removes the highlight and mark the step as complete.
- **Rationale**: Firefly needs to know when to move on from the highlight, as a highlight staying on the screen is confusing and not useful. For instructions with multiple steps, Firefly needs to know when one step is finished before showing the next. A rephrased instruction from the helper should also replace an old highlight.
- **Test Method:** Display a highlight and trigger each of the three conditions separately, which are click inside the highlighted region, click the dismiss control, and wait 15 seconds. Verify that the highlight is removed within 500 ms and the step is marked complete in all 10 attempts. Repeat this experiment with multistep commands.
- **Supporting Context:** A step is a singular highlight that gets generated so the learner can act on it. Clicking inside a highlighted region should be detected by reading the mouse position not by sending any input to process.
- **Trace**: SYS-04 Completion Detection
- **Priority**: Should Have

FR-13
- **SRID**: SYS-05
- **ID**: FR-13
- **Title**: Webcam Capture
- **Statement**: When the camera mode is enabled on iOS or through the webcam, Firefly captures the video at a minimum of 15 frames per second to pass to the object detection step within 100ms.
- **Rationale**: The webcam or camera is needed for live view guidance for physical objects. This way, the highlight can continuously follow the object.
- **Test Method:** Run a 1 minute capture and verify that each frame reaches the detection step within 100 ms and that the minimum FPS is 15. Then, unplug the camera and see if Firefly returns a camera not found message.
- **Supporting Context**: Real life Camera mode when firefly uses a camera instead of a screenshot for assistance with real life items such as a printer.
- **Trace**: SYS-05 Video Web Cam Capture
- **Priority**: Must Have

FR-14
- **SRID**: SYS-05
- **ID**: FR-14
- **Title**: Object Detection

- **Statement**: When camera mode is active, physical objects should be detected using computer vision, and Firefly will find the object named in the instruction at minimum 75% of frames when it is visible
- **Rationale**: The target has to be found in the frame before it can be highlighted. The system should detect general object types, such as a printer or a remote, so it works on unfamiliar models.
- **Test Method:** Capture 30 different frames of 5 common objects, such as a printer, router, or thermostat with buttons showing. Give an instruction that names the object, and verify that in 75% of frames, the button or object itself is detected. Then try 10 frames with blurry or dark lighting, and verify that Firefly asks for better adjustment. Lastly, capture 10 frames where the desired object is not visible and that Firefly returns that the instruction is not possible.
- **Supporting Context:** Computer vision finds objects based on certain boundaries.
- **Trace**: SYS-05 Process Frames Using Computer Vision
- **Priority**: Must Have

FR-15
- **SRID**: SYS-05
- **ID**: FR-15
- **Title**: Bounding Box Tracking
- **Statement**: After a new frame is processed, Firefly should update the highlight position to follow the tracked object, and keep the highlight within 10% of the object's center in at least 90% of frames.
- **Rationale**: Since the learner may not have a stable hand or may move around, the highlight should move with the object at times. If the camera drifts but the highlight doesn't drift with it, the highlight will become misleading.
- **Test Method**: Track a known object in view with a highlight for 30 seconds and record the video. Then using video editing software, verify the highlight stays within the 10% tolerance in at least 90% of frames. Then remove the object and verify that the highlight disappears within 1 second. Repeat with 5 identifiable objects.
- **Supporting Context**: Tracking is following the same object in every frame that the object is in view. Lag is the time that the highlight takes to update to be back within tolerance.
- **Trace:** SYS-05 Update Bounding Box as New Frames Are Processed
- **Priority**: Should Have

NFR-01
- **SRID**: SYS-04
- **ID**: NFR-01
- **Title**: Response Latency

- **Statement**: When a screenshot or image is taken along with an audio instruction that is within the range of 0-10 sub instructions, the system shall generate a highlighting element and animation on the learners device within 10 seconds after the audio instruction was given.
- **Rationale**: Maintaining fast response times ensures task completion efficiency and further incentives both learners and helpers in using Firefly.
- **Test Method**: Provide 30 different instructions, each with various amounts of associated sub instructions (1 to 10 sub instructions). Record time it takes for each instruction. Create a time versus sub instruction amount analysis and determine if each amount of sub instructions falls within the boundary. If it does not, determine the sub instruction number boundary that ensures a 10 second display time.
- **Supporting Context**: Sub instructions are defined as the individual screen actions, such as swipe up or swipe down, that go into completing a larger instruction.
- **Trace:** SYS-04
- **Priority**: Should Have

NFR-02
- **SRID**: SYS-03
- **ID**: NFR-02
- **Title**: Achieve reasonable target bounding box accuracy
- **Statement**: While detecting a certain object on screen, Firefly shall have a 90% accuracy rate when detecting objects and should have a bounding box no larger than a 5% swing in box size compared to the smallest box that can contain the object
- **Rationale**: Creating an accurate object detection system will provide the user with accurate information to help them learn, leading to a more positive learning experience than if the boxes from the vision were inaccurate or not useful.
- **Test Method**: Given 10-20 random objects the system needs to detect, the system must detect the object with 90% accuracy. This can be measured by if the system detects certain objects in various lightings and angles. While doing this test, if the object is detected the bounding box size will be measured. If the area of the box is -5-5% in size compared to the smallest area we find to contain the object at that angle, the system will pass for that object.
- **Supporting Context**: Bounding boxes are the boxes that the computer vision system will output on screen to show where a certain object is and how big it is.
- **Trace:** SYS-03 Target Identification
- **Priority**: Must Have

NFR-03
- **SRID**: SYS-01
- **ID**: NFR-03
- **Title**: Firefly will not store screenshots or captured audio after processing

- **Statement**: Firefly shall process all captured audio buffers, screenshots, and webcam frames strictly in volatile memory (RAM) and shall overwrite or free those buffers within 2 seconds after target coordinates are returned, leaving zero raw audio or image files saved on disk.
- **Rationale**: Learners often ask family members for help with sensitive tasks like online banking, medical portals, or private emails. As stated in our assumptions, personal information might briefly appear on screen or in audio, so Firefly must never write those screenshots or voice recordings to hard drive storage.
- **Test Method**: Run a 15-minute Firefly session containing 10 spoken instructions and 10 screenshot/webcam captures. Monitor the application's local AppData, temp directories, and working folders using Windows Process Monitor during the session, and inspect the filesystem afterward to ensure nothing is downloaded permanently.
- **Supporting Context**: For an instruction to have been processed means that the highlight has been displayed or the request itself has failed.
- **Trace:** SYS-01 Speech Capture and Transcription
- **Priority**: Must Have

NFR-04

- **SRID**: SYS-01
- **ID**: NFR-04
- **Title**: Clear Start-Up
- **Statement**: During initial launch of Firefly, it shall guide the learner through the startup process with no more than 5 steps, so that a first time user can complete the process in under 5 minutes.
- **Rationale**: Learners and novice users are easily put off by complicated setup, which is part of the reason why this software needs to exist in the first place. The app doesn’t matter if it cannot be set up properly.
- **Test Method**: Ask 10 first-time testers, ideally adults aged 60 or older, to install and start Firefly using only the on-screen instructions. Time the process to ensure it is under five minutes.
- **Supporting Context:** Startup covers granting the permissions Firefly needs (microphone or audio capture and screen or camera access) and turning it on. The learner should not need any help in setup. The 5 minute and 5 step limits are adjustable based on the trials.
- **Trace**: SYS-01 Speech Capture and Transcription
- **Priority:** Should Have

NFR-05

- **SRID**: SYS-04
- **ID**: NFR-05
- **Title**: Highlights and transcription should be presented in an accessible way to accommodate for visual impairments

- **Statement**: If Firefly displays a guide box or text instructions on the screen, then the Web/Mobile Client shall enforce a brightness contrast score of at least 7:1 for text and shall pair all colored highlights with a secondary non-color visual cue (such as a pulsing animation or an arrow).
- **Rationale**: Older adults frequently deal with reduced contrast sensitivity, glare, or color vision deficiencies. A thin yellow box disappears on a white webpage, and a solid red box blends into dark mode; adding a high-contrast dual border and a gentle pulsing animation makes sure the learner can spot the highlight right away regardless of their vision or screen theme.
- **Test Method**: We'll test the Firefly overlay on the hardest possible screens to read: a completely blank white screen, a pitch-black screen, and a heavily cluttered app. We'll use a contrast checker tool to prove the brightness differences hit those 4.5 and 7.0 scores. Then, we'll run screenshots of the app through a color-blindness simulator. If the boxes and arrows are still obvious when you strip away the original colors, the test passes.
- **Supporting Context**: The 4.5 and 7.0 scores come directly from standard web accessibility rules. A dual-border highlight (a bright inner color wrapped in a dark outer outline) usually guarantees the box will pop against any background.
- **Trace:** SYS-04 Highlight Rendering and Animations
- **Priority**: Must Have

5. Deliverables

Deliverable 1: Deployable Windows Desktop Application & Installer Package (Firefly-Setup.exe)

- A standalone executable and installer that packages the complete Firefly learner experience—including automatic system loopback/microphone audio capture, the local speech-to-text transcription engine, the transparent click-through screen overlay window, and the OpenCV webcam viewing window—so a novice user can install and start the app in two clicks without installing Python or developer dependencies manually.
- FR-01 Audio Input Capture, FR-02 Audio Pre-Processing, FR-03 Speech Transcription, FR-10 Highlight Overlay Rendering, FR-11 Non-Interactive Overlay, FR-12 Completion Detection, FR-13 Webcam Capture, NFR-04 Usability

Deliverable 2: Accessibility, Pass-Through Safety, and Privacy Audit Report

- A verification test demonstrating that Firefly is safe and accessible for older adults. This report will include WebAIM Contrast Checker results and color-blindness simulation screenshots for our highlight overlays, OS input-log proofs and static code scans confirming zero mouse/keyboard injection (SendInput), and Windows Process Monitor

(ProcMon) filesystem logs proving that screenshots and call audio are wiped from RAM and never written to disk.
- FR-07 Capture Field of View, FR-11 Non-Interactive Overlay, NFR-03 Privacy & Storage, NFR-05 Accessible Presentation

Deliverable 3: Containerized AI Target Identification & Vision Pipeline
- A packaged backend module containing the multimodal target identification model and the OpenCV object tracking logic. By separating it, we can swap out better vision models later without breaking the whole app. This module takes the captured screenshots or webcam frames, processes the spoken text, and returns the normalized bounding boxes.
- FR-04 Instruction Identification, FR-05 Instruction Boundary Detection, FR-06 Sequential Instruction Processing, FR-07 Capture Field of View, FR-08 Target Identification, FR-09 Bounding Box Return, FR-14 Object Detection, FR-15 Bounding Box Tracking

Deliverable 4: End-to-End Latency and Target Accuracy Benchmark
- A quantitative testing report validating the speed and correctness of the entire Firefly pipeline. We can't assume an AI model is fast enough for a live video call. This deliverable holds our automated benchmark scripts and the raw data from testing 30 different UI screens and physical objects. It proves the system hits at least 75% bounding box accuracy and fully renders the highlight within a reasonable time frame.
- FR-03 Speech Transcription, FR-08 Target Identification, FR-14 Object Detection, NFR-01 Latency & Time Frame, NFR-02 Bounding Box Accuracy

Deliverable 5: Plain-Language User Setup Guide & Developer Architecture Package
- A two-part documentation package: 1) a single-page, large-print (18pt+) visual guide written in English and Spanish showing learners how to open Firefly during a Zoom or phone call, and 2) developer documentation containing our system architecture diagrams, model prompt/API schemas, OpenCV pipeline documentation, and build instructions for future maintainers
- FR-01 Audio Input Capture, NFR-04 Usability

6. Standards (1 person task)

6.1 ISO/IEC/IEEE 29148-2018

Standard: International Standard - Systems and software engineering, Life cycle processes, Requirements engineering

Link: IEEE SA - IEEE/ISO/IEC 29148-2018

Project Application: This standard is specifically applicable to our project because of the nature of a software project, Firefly will most likely have many iterative and recursive requirement adaptations through the development process.

Project Impact: This will likely impact how we go through revisions and how we evaluate our requirements. More specifically when using GitHub issues we can document requirement changes and track their progress in accordance to the listed practices.

6.2 ISO/IEC 29100:2024

Standard: Information technology, Security techniques, Privacy framework

Link: ISO/IEC 29100:2024 - Information technology — Security techniques — Privacy framework

Project Application: This standard is applicable to our project because we might obtain or deal with sensitive data. This will provide a reference for privacy principles and important safeguarding considerations.

Project Impact: This will likely impact our code design, and more specifically how we collect, store, and manage data. This will mean we will ensure temporary data is deleted after all audio and image processing is completed.

6.3 ISO/IEC 25010:2023

Standard: Systems and software engineering, Systems and software Quality Requirements and Evaluation (SQuaRE), Product quality model

Link: ISO/IEC 25010:2023 - Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model

Project Application: This standard will be specifically relevant in our project when we start testing and evaluating our product. Using criteria for products, quality control, and measure of product quality will be good references.

Project Impact: This will likely impact our testing pipeline and how we evaluate success in a software context. More specifically, we will develop test cases for all of our requirements and evaluate how accurate and successful they were.

7. GenAI Statement

We used GenAI to help guide discussion in breaking down components and cscope. We wrote all writing ourselves.